

CLI Tools (Gemini CLI and GitHub Copilot CLI)

Contents

Install	1
Quick Start (Gemini CLI)	1
1) Start a context-aware session	1
2) Common tasks	2
Use These Assets With Gemini CLI	2
Option A (recommended): project-local setup	2
Option B: user-wide skills	3
Included Gemini commands	3
Included Gemini skills	3
Quick Start (GitHub Copilot CLI)	3
Common tasks	3
About agents and skills for Copilot	4

This repo is a set of ready-to-copy assets for working with Gemini CLI:

- Project instructions (`GEMINI.md`)
- Gemini CLI settings (`.gemini/settings.json`)
- Custom slash commands (`.gemini/commands/*.toml`)
- Skills (`.gemini/skills/**/SKILL.md`)

See also: `gemini-readme.md` for extra Gemini CLI notes (extensions, Conductor, etc.).

A smaller section at the end covers GitHub Copilot CLI.

Install

For installation see the following:

- Gemini CLI: <https://geminicli.com/>
- GitHub Copilot CLI (via GitHub CLI): <https://docs.github.com/>

Quick Start (Gemini CLI)

1) Start a context-aware session

Run Gemini CLI from the root of the project you want it to work on:

```
cd /path/to/your/project
gemini
```

2) Common tasks

Use natural language prompts (or your own `/commands` if configured).

- Understand a repo:
 - “Summarise this repository: main components and entry points.”
- Explain code:
 - “Explain what `src/foo/bar.ts` does and how it’s used.”
- Plan work before coding:
 - “/plan Add caching to the API client and tests”
- Review a doc:
 - “Review this markdown for clarity and broken links: ...”

Tip: if your Gemini CLI setup supports shell passthrough, you can run commands from inside a session (commonly via `!<command>`). If unsure, use `help` or `/help` in the CLI.

Use These Assets With Gemini CLI

The templates in this repo live under `files/gemini/` and are meant to be copied into either:

- a specific project (recommended), or
- your user config (if you want them everywhere).

Option A (recommended): project-local setup

From your target project directory:

```
export CLI_TOOLS_DIR=/path/to/this/repo

mkdir -p .gemini/commands .gemini/skills

# Project instructions (loaded when you run `gemini` in this folder)
cp "$CLI_TOOLS_DIR/files/gemini/GEMINI.md" ./GEMINI.md

# Gemini CLI settings for this project
cp "$CLI_TOOLS_DIR/files/gemini/settings.json" ./gemini/settings.json

# Custom slash commands (invoked like: /plan <goal>)
cp "$CLI_TOOLS_DIR/files/gemini/commands/*.toml" ./gemini/commands/

# Skills (discovered by Gemini CLI)
cp -r "$CLI_TOOLS_DIR/files/gemini/skills/*" ./gemini/skills/
```

What you get:

- Instructions: GEMINI.md
- Commands: .gemini/commands/plan.toml, .gemini/commands/dokuwiki.toml, .gemini/commands/gnur-programming.toml
- Skills: .gemini/skills/**/SKILL.md

Option B: user-wide skills

Gemini CLI can also discover skills from a user directory.

```
export CLI_TOOLS_DIR=/path/to/this/repo
```

```
mkdir -p ~/.gemini/skills
```

```
cp -r "$CLI_TOOLS_DIR/files/gemini/skills/"* ~/.gemini/skills/
```

Gemini skill precedence is typically:

1. Project skills: .gemini/skills/
2. User skills: ~/.gemini/skills/
3. Extension skills

If multiple skills share the same name, project-local skills win.

Included Gemini commands

These ship as templates in `files/gemini/commands/`:

- `/plan <goal>`: planning-only mode (no code changes)
- `/dokuwiki <content>`: Dokuwiki helper / reviewer
- `/gnur <content>`: GNU R programming review helper

Included Gemini skills

These ship as templates in `files/gemini/skills/`:

- document critique
- dokuwiki validator
- haskell programmer
- markdown validator
- python programmer

See also: `files/gemini/gems/` for reusable prompt “gems”.

Quick Start (GitHub Copilot CLI)

Copilot CLI is typically exposed via GitHub CLI as `gh copilot`.

Common tasks

```
# Ask for a command suggestion
```

```
gh copilot suggest "find large files in this repo"
```

Explain a command you don't understand

```
gh copilot explain "tar -xzf archive.tgz -C /usr/local"
```

Discover what else is available

```
gh copilot --help
```

```
gh copilot suggest --help
```

```
gh copilot explain --help
```

About agents and skills for Copilot

The Copilot CLI itself does not typically consume agent persona files. If you are using Copilot Agents (e.g., in VS Code), this repo includes:

- Agent prompts in `files/code/prompts/`: copy to `~/.config/Code/User/prompts`
- Skill templates in `files/code/skills/`
 - repo-scoped: copy into `.github/skills/`
 - user-scoped: copy into `~/.copilot/skills/`